10-01 / 10-08: top-down parsing

Lexical categories are simple, regular sub-languages of the programming language. Regular expressions and DFAs are great for defining these and checking membership, but programming languages (i.e., programs) tend to be non-regular; in particular, the syntactic (grammatical) rules describing the way the lexical symbols form bigger structures (such as expressions or statements) is typically not regular. This means that the syntax of most programming languages cannot be defined with a regular expression (or Chomsky-3 grammar), but it can be defined with a more expressive formalism, a context-free grammar (Chomsky-2).

Syntax is context-free

Take for example a language (over the alphabet {*a,b*}) that contains words composed of a sequence of *a*'s and a sequence of *b*'s, where the two sequences are of the same length. Without the last condition, the language would be regular and could be described by the regular expression *a*b*;* however, ensuring the length constraint requires a context-free rule that can generate *a*'s and *b*'s simultaneously:

```
S -> ε | a S b
```

In the above grammar, *S* is a non-terminal (a grammar symbol), *a* and *b* are terminals (alphabet symbols), and ε (epsilon) is the empty word. For brevity, from now on we'll assume that capital letters denote non-terminals and lowercase letters denote terminals (and if not defined explicitly otherwise, *S* is the start symbol).

Remember: both regular and context-free grammar rules' LHS is a nonterminal, but while regular rules cannot generate terminals to both sides of a nonterminal in the RHS, context-free rules are effectively unconstrained (the RHS is an arbitrary sequence of terminals and non-terminals). The rule *A->aB* is regular, whilst *A->aBc* is context-free (allowing the LHS to be rewritten/replaced regardless of its context). However, bear in mind that since the LHS of any context-free rule remains a non-terminal, terminal symbols in sentential forms cannot be rewritten by context-free rules either --- this is an important property we will exploit when designing efficient parsing algorithms.

Derivations

A context-free grammar generates a context-free language. Sentences of the language are terminal sequences derived from the start symbol by using the production rules for rewriting non-terminals. For example, a derivation in the above grammar would be:

```
S => aSb => aaSbb => aaaSbbb  => aaabbb
```

By using the second rule twice, then the first rule once, we generated the sentence *aaabbb*. The intermediate sequences are called sentential forms; in particular, the start symbol is a sentential form as well as sentences are sentential forms (latter only containing terminal symbols).

Derivations of sentences determine syntax trees representing the structure of the sentence:

```
   S
  /|\
 a S b
  /|\
 a S b
  /|\
 a S b
   |
   ε
```

A syntax tree's root is always labelled with the start symbol, internal nodes are labelled by non-terminals, and leaves are labelled by terminals. A single-level subtree is always labeled such that the parent and children are the LHS and RHS symbols of a production rule, respectively. Reading the "front" of the tree (the leaves from left to right) determines the sentence represented by the tree.

When parsing, we do not only want to *recognize* the language accepted by the grammar, but we also intend to *build the parse tree* for recognized sentences. The input to parsing is the sequence of terminal symbols (the lexical symbols yield by the scanner), and the output is either the parse tree, or the syntax errors.

Leftmost and rightmost derivations

Derivations are not necessarily unique. Production rules can generate multiple non-terminals in sentential forms and thus allow for an arbitrary order of rewriting. For example, the grammar

```
S -> A A
A -> a
```

has two different derivations for the sentence *aa*:

```
S => AA => aA => aa
S => AA => Aa => aa
```

The first one is called the left-most derivation, the second one is called the right-most derivation: the former rewrites/expands the left-most nonterminal in each derivation step, while the latter rewrites the right-most nonterminal. Yet, they represent the same syntax tree. Top-down syntax analysis techniques tend to yield the left-most derivation, while bottom-up techniques produce a right-most derivation.

Ambiguity

In some grammars, different derivations of a sentence may represent different syntax trees, resulting in ambiguous derivations and therefore grammar ambiguity. For example,

```
S -> a | S + S
```

The sentence *a+a+a* has two entirely different derivations: one representing a binary tree leaning to the

left, and another one leaning to the right. The derivations can be written like follows:

```
S => S+S => S+a => S+S+a => S+a+a => a+a+a
S => S+S => a+S => a+S+S => a+a+S => a+a+S
```

And the syntax trees would look like this:

```
    S
   /|\
  S + S
 /|\  |
S + S a
|   |
a   a
```

and this:

```
    S
   /|\
  S + S
  |  /|\
  a S + S
    |   |
    a   a
```

As you can imagine, the problem with such grammars is that the compiler (the syntax analyser) has to make a choice between the options, and in the worst case, different runs may yield different trees, thus different structures --- and possibly different meaning at the end of the compilation. We avoid non-deterministic grammars and parsing in the process and make sure every program is mapped to a unique syntax tree.

But there's a catch: parser generators intentionally allow grammar developers to write definitions like the above; however, they also allow the developer to make the decision between the two options, declaring the associativity of the operator. In bisonc++, you can use the directives *%left* and *%right* to enforce the former or the latter options.

S-expressions

This issue with generating the same number of *a* and *b* symbols resembles the problem of generating expressions (or rather, recognizing expressions) with balanced parenthesis. Indeed, a slightly more complex grammar defines S-expressions, which represents the complexity of the average programming language syntax:

```
S → x | ( A )
A → ε | S A
```

This grammar generates sentences like:

```
x, (), (x), (x())
```

Let us construct the derivation for the latter one:

```
S => (A) => (SA) => (xA) => (xSA) => (x(A)A) => (x()A) => (x())
```

This is a left-most derivation. Top-down parsers construct such a derivation for the input sequence. (Note that the terminals in this case are in fact the tokens recognized by the scanner.)

How do top-down parsers do such a derivation? Well, simply, in every sentential form, they find the left-most non-terminal, and rewrite it with one of the production rules. The question worth investigation is how the rule is chosen (or not chosen).

Naively, one can do backtracking (try the possible rewrites, observe when the sentential forms' terminal prefix does not match the input's prefix, and try a different derivation), but that, although very general, is very costly. You have learned about the CYK algorithm in the FOTOC course, that can parse sentences of all context-free grammars expressed in CNF (Chomsky normal form), but that is a cubical algorithm. **Can we restrict the grammar so that the parsing can be done in linear time, reading the input once from left to right, possibly with some lookahead?**

LL(1)

One potential answer to the above question is LL(k) parsing. A grammar is LL(k) if and only if the first *k* terminal symbols determine which production rule of a nonterminal must be applied. LL (reading left-to-right, producing left-most derivation) parsers are predictive, and they predict their next step based on a lookahead sequence (which is typically of length 1).

LL(1) predictive top-down parsing is based on a number of basic ideas:

- When working with context-free grammars, terminals cannot be rewritten in derivation steps; that is, if a sentential form has a terminal prefix, it cannot be changed via derivation.
- Rewriting the left-most nonterminal in each derivation step ensures that terminal symbols appear on the left of the sentential form, in the prefix, and hence can be matched to the input.
- If we restrict context-free grammars, the rule to be used to rewrite the leftmost nonterminal can be deterministic, allowing for a deterministic, predictive parsing algorithm.
- The prediction method can be explained as a table that determines the next derivation step based on the leftmost nonterminal and the lookahead symbol of the input.

Intuition on predictive top-down parsing

In the above derivation example with the S-expression grammar, we made nontrivial decisions about the usage of the production rules. However, a deterministic decision process can be established, based on what's the next terminal symbol in the remaining part of the input. More precisely, in the example derivation, the first sentential form is *S*, while the input starts with an opening parenthesis; this suggests that *S* should be rewritten with a rule that produces an opening parenthesis, and indeed, the second rule does that. Then, the opening parenthesis we just generated matches the input, so in the result sentential

form we rewrite the non-terminal *A* to produce an *x* symbol (which is the current lookahead after accepting the opening parenthesis). In this case, we need further investigation as neither of *A*'s rules starts with an *x*; however, the alternative *SA* allows the *S* to be rewritten to *x*, thus generating the required symbol in a subsequent step. Whenever we have to rewrite the leftmost nonterminal, we try to figure out which alternative has the chance to actually generate the due part of the input sequence.

We continue using the same intuition until the sentential form *(x(A)A)*. Here we fail to use the same idea as none of the rules of *A* produce a closing parenthesis. Indeed, the right decision to be made here is rewriting *A* to epsilon (that is, nothing) and make it disappear. What's the terminal generated by this rule? None. However, we can predict what terminal *A* will be replaced by: the symbol that appears after it. Since in the rule *(A)* the symbol *A* is followed by the closing parenthesis, we can speculate that getting rid of A will actually produce a closing parenthesis (or in other words, after rewriting *A* to epsilon we get a closing parenthesis at the position of the now-gone *A* symbol). This comes from this very observation that in sentential forms, *A* will be followed by the closing parenthesis symbol. Apparently, whenever the above technique does not suggest any of the rules to be applied, the parser has to yield a syntax error.

In the following, we will define FIRST and FOLLOW sets, which are to be used to "predict" the terminals produced by the rule alternatives. Based on these, we draw a parsing table (mapping the left-most nonterminal and lookahead terminal to an action), and explain how the table-based algorithm works.

LL(1) in detail

Let's take the S-expression grammar, replace the parenthesis terminal symbols by *a* and *b*, and assign identifiers to each rule for references:

```
S → x | a A b
    1     2
A → ε | S A
    3     4
```

We built the intuition behind the predictive parsing of the sentence *axabb*, mostly based on the prediction of which rules produce which terminals symbols. The predictions were particularly interesting in two cases:

- Explaining when the *A->SA* rule must be used for derivation, we argued about what *S* can be rewritten to. Indeed, this leads us to the concept of FIRST sets, which determine the terminal symbols that can be derived from a symbol of a sequence of symbols.
- Explaining when the epsilon-rule is to be used for derivation, we argued about what terminals may appear in place of the left-hand nonterminal, when being rewritten to epsilon. This leads us to the concept of FOLLOW sets, which determine the terminal symbols that can appear after the left-hand nonterminal in sentential forms.

FIRST and FOLLOW sets

The definitions of FIRST and FOLLOW, and the algorithms for computing them, can be found in the **Stanford handout** **(https://web.stanford.edu/class/archive/cs/cs143/cs143.1128/handouts/090%20Top-**

Down%20Parsing.pdf) (page 9) and in the **Cornell handout** (https://www.cs.cornell.edu/courses/cs4120/2026sp/notes.html?id=topdown) (although latter does not include the epsilon in the FIRST sets but maintains a separate NULLABLE set).

For the grammar above, the right-hand sides have the following $FIRST_1$ sets:

```
(rule 1) FIRST(x) = {x}
(rule 2) FIRST(aAb) = {a}
(rule 3) FIRST(ε) = {ε}
(rule 4) FIRST(SA) = FIRST(S) = FIRST(x) U FIRST(aAb) = {x,a}
```

Note that although the Cornell handout does not include the epsilon in the FIRST set, in this regard we follow the Stanford handout and do include epsilon in the FIRST set whenever the symbol sequence is empty or it can derive the empty word (i.e. it is NULLABLE).

In the case of the epsilon-rule, according to the above explanation, we need to also compute the FOLLOW set of *A*:

```
FOLLOW(A) = FIRST(b) = {b}
```

This suggests that we shall use the epsilon-rule whenever the lookahead symbol is a *b* terminal. This matches our intuition perfectly, since whenever the *A* symbol appeared in a sentential form, it was followed by a *b* that was generated by *S->aAb*.

When computing these FIRST and FOLLOW sets, we perform a basic static analysis of the context-free grammar, in order to determine which terminal symbols "trigger" which production rules. Apparently, whenever a single terminal would trigger multiple production rules of a nonterminal, the grammar cannot be LL(1)-analysed and therefore we say it is not an LL(1) grammar (the parsing would be branching or backtracking).

The LL(1) parsing table

Now having computed the terminal sets that trigger the rules, we can summarise them in a simple tabular, the so-called LL(1) parsing table. The rows are labelled by non-terminals (which are to be expanded) and the columns are labelled by the potential lookahead symbols (terminals, and the end-of-string symbol, #). A rule is written in a cell in the row of its LHS whenever the column's label (the lookahead symbol) is included in the FIRST set of the RHS (or in the FOLLOW set of the LHS, if the RHS is nullable).

```
  |  x   a   b   #
------------------
S |  1   2
A |  4   4   3
```

If any of the cells contains multiple rule identifiers, that signals a parser conflict and therefore the grammar is not LL(1) parsable. For example, if we added a production rule *A->b*, it would clash with the epsilon-rule, making it non-LL(1). You can check this yourself, or feel free to inspect parsing tables by

using **Grammophone** (https://mdaines.github.io/grammophone/) .

The LL(1) parsing algorithm

Based on the table, we use the following algorithm for parsing. We have a configuration consisting of a stack of the postfix of the current sentential form (the symbols expected to appear on the input), and the postfix of the input. The terminal symbols matched already are dismissed, popped from the stack. In each step, the parser checks the top of the stack (the leftmost unaccepted symbol from the sentential form) and the first symbol of the rest of the input (the lookahead) and takes one of the following actions:

- If the stack is empty and the lookahead is the end-of-string symbol, **accept** the input.
- If the top of the stack is a terminal, and it is **matched** by the lookahead symbol, the symbol is popped from the stack, and the input is advanced.
- If the top of the stack is a terminal, but it is not matched by the lookahead symbol, signal a syntax **error**.
- If the top of the stack is a non-terminal and the parsing table prescribes the application of a production rule, **apply** the rule: the LHS is popped from the stack and the RHS symbols are pushed to the stack (in reverse order). The input remains intact.
- If the top of the stack is a non-terminal, but the parsing table does not prescribe the application of a production rule, signal a syntax **error**.

Parsing example

The following snippet shows the LL(1) parsing of the above-mentioned *axabb* example string, using the LL(1) parsing table. Each line takes a single action with the analyser, the action is written at the end of the line.

```
(S#,      axabb#) -> (2)
(aAb#,    axabb#) -> pop a
(Ab#,      xabb#) -> (4)
(SAb#,     xabb#) -> (1)
(xAb#,     xabb#) -> pop x
(Ab#,       abb#) -> (4)
(SAb#,      abb#) -> (2)
(aAbAb#,    abb#) -> pop a
(AbAb#,      bb#) -> (3)
(bAb#,       bb#) -> pop b
(Ab#,         b#) -> (3)
(b#,          b#) -> pop b
(#,            #) -> accept
```

Reading the production rule applications from top to bottom gives a left-most derivation:

```
Leftmost derivation of "axabb": 2,4,1,4,2,3,3
```

Suggested exercise: do a similar parsing for the input "axbb" and see when and how the error appears.

Recursive descent parsing implementation

One of the most straightforward and simplest implementations of LL(1) parsing is the so-called recursive descent technique, which utilises the run-time stack of a program to store the parsing stack (containing the due part of the current sentential form). In particular, whenever a production rule is applied, its RHS symbols are not pushed onto a stack, but corresponding methods are invoked: the symbols "are pushed onto the stack", or more precisely, the activation records of methods of the same name are pushed onto the stack. Whenever the symbols "belonging" to a nonterminal are accepted, its method's body terminates, and therefore it gets removed from the stack.

For each non-terminal, the recursive descent parser defines a method, and for terminals, it defines a general method that checks if it matches the lookahead. The methods belonging to the non-terminals can use the lookahead to decide what symbols (i.e. which RHS) they push onto the stack.

The only major difference from the LL(1) algorithm discussed above is that upon applying a production rule (and therefore pushing the RHS symbols onto the stack), the recursive descent parser does not remove the LHS from the stack and the RHS symbols are not pushed at once in reverse order. Rather, calling A() pushes a stack frame, and that frame will not be popped until A() fully returns. This means that all of A's own sub-parsing completes entirely before control ever returns to the LHS. So the call stack at any instant reflects the current path from the root to the active symbol (like a vertical slice of the parse tree), not the "remaining symbols to process" the way the explicit LL(1) stack does. Since each RHS symbol is called first and must fully complete (return) before the next is even invoked, the natural sequential nature of function calls enforces left-to-right order automatically.

A recursive descent parser for the above-discussed S-expr grammar can be found **here (https://canvas.elte.hu/courses/66503/files/4501005/download?wrap=1)** , implemented in C++.

Further topics to visit

- LL(k) parsing with k-long lookahead (more lookahead symbols may make better predictions)
- Context-free grammars that are not LL(k) for any k
- Left-recursion and its elimination
- Epsilon rules versus nullable rules
- Adaptive LL used in the ANTLR parser generator